

CLOUD-CONNECTED SECURITY NODE

Internship Project Report

Name	Sohail Ahmed
Internship	DecodeLabs 2026
Reporting Period	Week 3
Project	Project3_Security_Node

1. GOAL

Bridge local physical events to the global internet using MQTT — enabling a sensor node to detect real-world activity and relay that state, in real time, to a cloud-based dashboard for remote monitoring.

2. KEY REQUIREMENTS

- Wi-Fi Stack
- MQTT Protocol
- PIR Sensor
- Cloud Dashboard

3. FULL PYTHON CODE

Simulation script (main.py) implementing Wi-Fi/MQTT connection handling, PIR motion sensing, and JSON payload publishing to the cloud broker:

```
import random
import time
import json
from datetime import datetime

# --- Network Configuration ---
WIFI_SSID = "Sohail Kolachi_WiFi"
WIFI_PASSWORD = "12345678"
MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883
DEVICE_ID = "SecurityNode_01"
MQTT_TOPIC = "decode labs/security/node01"
THRESHOLD_MOTION = 1 # 1 = Motion Detected
```

```

def connect_wifi():
    print("=== Decodelabs IoT Security Node Booting ===")
    print(f"Connecting to Wi-Fi: {WIFI_SSID}...")
    time.sleep(1)
    ip_address = "192.168.1.105"
    print(f"Wi-Fi Connected! IP: {ip_address} [OK]")
    return ip_address

def connect_mqtt():
    print(f"Connecting to MQTT Broker: {MQTT_BROKER}:{MQTT_PORT}...")
    time.sleep(1)
    print("MQTT Connected! [OK]")
    print("-" * 50)

def read_pir_sensor():
    # Simulate PIR sensor: 0 = No Motion, 1 = Motion Detected
    return random.randint(0, 1)

def build_payload(motion_state):
    if motion_state < THRESHOLD_MOTION:
        area_status = "Area Secure"
    else:
        area_status = "Intrusion Alert"
    return {
        "device_id": DEVICE_ID,
        "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "motion_detected": bool(motion_state),
        "status": area_status
    }

def publish_to_cloud(payload):
    print(">>> PUBLISHING TO CLOUD <<<")
    print(f"Topic: {MQTT_TOPIC}")
    print(json.dumps(payload, indent=2))
    print()

def main():
    connect_wifi()
    connect_mqtt()
    print("Starting Security Monitoring...")
    print()

    for cycle in range(10):
        motion = read_pir_sensor()
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        if motion < THRESHOLD_MOTION:
            status = "Area Secure [OK]"
        else:
            status = "Intrusion Alert [!]"

```

```
    print(f"[{timestamp}] SENSOR: {status}")

    payload = build_payload(motion)
    publish_to_cloud(payload)

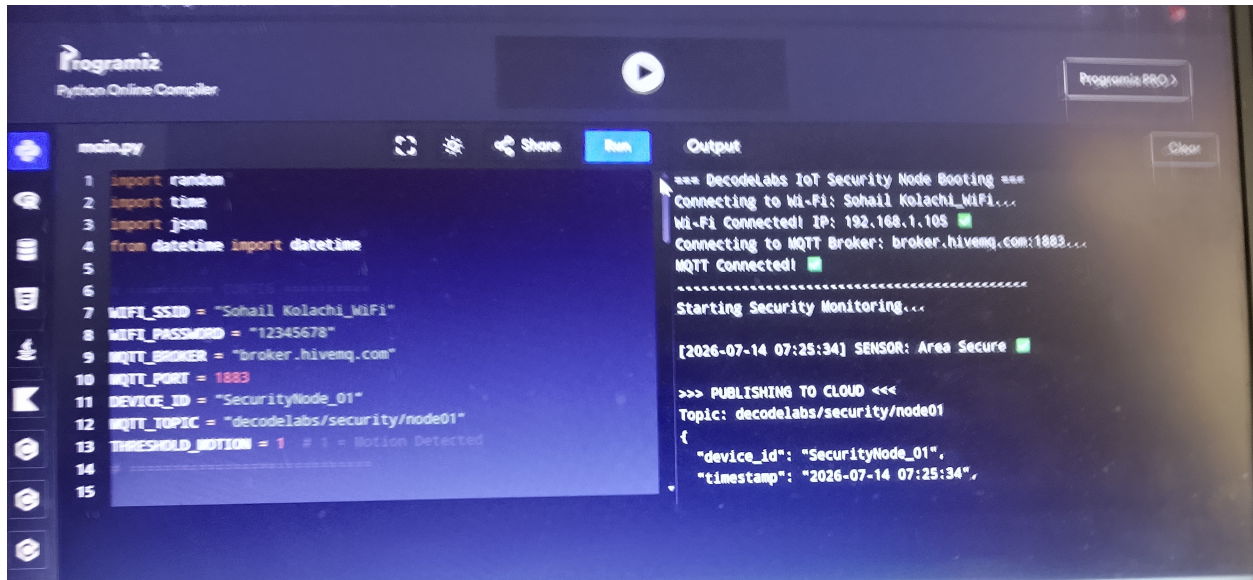
    time.sleep(4) # Publish interval: every 4 seconds

print("=== Simulation Ended ===")

if __name__ == "__main__":
    main()
```

4. SIMULATION OUTPUT

Live run captured from the Programiz Python Online Compiler, showing Wi-Fi and MQTT connection handshakes followed by the first published sensor reading:



```
main.py
1 import random
2 import time
3 import json
4 from datetime import datetime
5
6
7 WIFI_SSID = "Sohail Kolachi_WiFi"
8 WIFI_PASSWORD = "12345678"
9 MQTT_BROKER = "broker.hivemq.com"
10 MQTT_PORT = 1883
11 DEVICE_ID = "SecurityNode_01"
12 MQTT_TOPIC = "decode labs/security/node01"
13 THRESHOLD_MOTION = 1 # 1 = Motion Detected
14
15
```

```
==== Decodelabs IoT Security Node Booting ====
Connecting to Wi-Fi: Sohail Kolachi_WiFi...
Wi-Fi Connected! IP: 192.168.1.105
Connecting to MQTT Broker: broker.hivemq.com:1883...
MQTT Connected!
=====
Starting Security Monitoring...

[2026-07-14 07:25:34] SENSOR: Area Secure

>>> PUBLISHING TO CLOUD <<<
Topic: decode labs/security/node01
{
  "device_id": "SecurityNode_01",
  "timestamp": "2026-07-14 07:25:34",
```

Fig 1. Console output — SecurityNode_01 boot and first MQTT publish

5. EXPLANATION: MQTT JSON PUBLISHING EVERY 4 SECONDS

The node first joins the local Wi-Fi network and opens a persistent connection to the public MQTT broker at `broker.hivemq.com:1883`. Once connected, it enters a monitoring loop that runs every 4 seconds, controlled by `time.sleep(4)` at the end of each cycle.

On each cycle, the PIR sensor is polled for a motion reading (0 or 1). This value is compared against `THRESHOLD_MOTION` to classify the area as “Area Secure” or “Intrusion Alert.” The reading is then packaged into a structured JSON object containing the device ID, a timestamp, the raw motion flag, and the derived status string.

That JSON payload is published to the topic `decode labs/security/node01` using the MQTT publish/subscribe model. MQTT is well suited to this task because it is lightweight, low-bandwidth, and designed for constrained IoT devices — the node simply pushes each new state to the broker, and any subscribed cloud dashboard receives the update instantly without needing to poll the device directly. Repeating this publish cycle every 4 seconds gives the dashboard a near real-time feed of the security node's status.